Санкт-Петербургский государственный университет

СИНЕЛЬНИКОВ Михаил Алексеевич

Выпускная квалификационная работа

Реализация алгоритма FastDD для поиска дифференциальных зависимостей в рамках профилировщика данных Desbordante

Уровень образования: бакалавриат

Направление *02.03.03 «Математическое обеспечение и администрирование информационных систем»*

Основная образовательная программа *СВ.5162.2022 «Технологии программирования»*

Научный руководитель:
доцент кафедры информационно-аналитических систем, к. ф.-м. н., Михайлова Е. Г.

Консультант:
ассистент кафедры информационно-аналитических систем, Чернышев Г. А.

Рецензент:
Программист-разработчик, ООО «ВК», Слесарев А. Г.

Санкт-Петербург
2026

Saint Petersburg State University

Mikhail Sinelnikov

Bachelor's Thesis

Implementing the FastDD Algorithm for Discovering Differential Dependencies in the Desbordante Data Profiler

Education level: bachelor

Speciality *02.03.03 "Software and Administration of Information Systems"*

Programme *CB.5162.2022 "Programming Technologies"*

Scientific supervisor:
C.Sc, docent E.G. Mikhailova

Consultant:
Assistant, G.A. Chernishev

Reviewer:
Programmer-developer at LLC "VK", A.G. Slesarev

Saint Petersburg
2026

Оглавление

Введение **4**

1. Постановка задачи **6**

2. Обзор **7**

2.1. Дифференциальные зависимости 7

2.2. Алгоритм FastDD 13

2.3. Открытая реализация алгоритма 15

3. Описание решения **19**

3.1. Первоначальная реализация 19

3.2. Оптимизации и улучшения алгоритма 22

4. Эксперимент **33**

4.1. Время работы и потребляемая память 34

4.2. Зависимость времени работы от числа строк и атрибутов 35

4.3. Вклад оптимизаций алгоритма 38

4.4. Выводы 38

Заключение **40**

Список литературы **41**

Введение

Данные играют ключевую роль в современном мире. Компании и организации хранят и обрабатывают огромное количество данных: информацию о пользователях, протоколы внутренних операций, различные виды статистик и т.д. Информация, извлекаемая из данных, используется как для обеспечения бесперебойности доступа и повышения удобства использования сервисов пользователем, так и для принятия внутренних бизнес-решений компаний. Таким образом, эффективная обработка большого количества данных и извлечение полезной информации из них является важным навыком.

Задача получения информации из набора данных называется профилированием данных [1]. Результатом профилирования могут являться как простые статистики (количество строк в файле или типы данных, хранящихся в нём), так и более нетривиальные закономерности.

Одним из наиболее известных примеров таких нетривиальных зависимостей, извлекаемых из табличных данных, являются функциональные зависимости (FD) [6]. Говорят, что в таблице удерживается функциональная зависимость $X \to Y$ (X, Y — множества атрибутов таблицы), если для каждого уникального значения на атрибутах X существует ровно одно соответствующее уникальное значение на атрибутах Y. Функциональные зависимости используются для нормализации баз данных и улучшения качества данных.

Данные, встречающиеся на практике, зачастую могут содержать множество ошибок. Опечатки, неверные записи, различные форматы записи информации одного и того же типа делают данные неоднородными, что значительно осложняет их обработку. Часто большие объёмы данных являются результатом автоматической интеграции данных, полученных с помощью запросов к огромному числу веб-страниц или баз данных, что приводит к возникновению такого рода ошибок. Следствием этой глобальной проблемы является затруднительное использование функциональных зависимостей на практике [8]. Одним из подходов к решению данной проблемы в случае функциональных за-

висимостей являются теоретические обобщения понятия функциональной зависимости, учитывающие в некоторой степени неоднородность данных.

Дифференциальная зависимость (DD) [11] — одно из обобщений понятия функциональной зависимости, вычисляющее расстояние между записями для определения степени их различия. Для вычисления расстояния используются метрики, которые определяются для каждого атрибута, учитывая его тип данных. Например, метрикой для числовых данных может являться модуль разности значений, а для строковых данных — расстояние Левенштейна [9].

Desbordante[1] [2, 3] — высокопроизводительный наукоёмкий профилировщик данных с открытым исходным кодом, решающий задачи поиска и валидации нетривиальных типов зависимостей в табличных данных. Среди большого множества видов закономерностей, поддерживаемых в Desbordante, присутствуют функциональные и дифференциальные зависимости.

Мною в рамках Desbordante был реализован[2] [13] и оптимизирован[3,4] [15] алгоритм SPLIT поиска дифференциальных зависимостей, описанный в статье [11]. Однако в 2024 году в статье [4] был предложен новый алгоритм поиска DD под названием FastDD[5], а также произведено его сравнение[6] с алгоритмом SPLIT, показавшее, что новый алгоритм является значительно более производительным, чем оригинальный. Обзор, проведённый мной в отчёте [14] за весну 2025 года, подтвердил предпочтительность алгоритма FastDD перед алгоритмом SPLIT. Кроме того, в данном отчёте было показано, что алгоритм FastDD ищет DD согласно более узкому определению, чем SPLIT. Таким образом, возникла необходимость реализовать, а также оптимизировать новый алгоритм и расширить его для поддержки более общего определения в рамках Desbordante для повышения производительности поиска DD.

[1] https://github.com/Desbordante/desbordante-core
[2] https://github.com/Desbordante/desbordante-core/pull/374
[3] https://github.com/Desbordante/desbordante-core/pull/439
[4] https://github.com/Desbordante/desbordante-core/pull/506
[5] https://github.com/TristonK/FastDD
[6] https://github.com/TristonK/FastDD-Exp

1. Постановка задачи

Целью работы является реализация и оптимизация алгоритма FastDD для поиска дифференциальных зависимостей в рамках профилировщика данных Desbordante. Для достижения цели были поставлены следующие задачи:

- провести обзор алгоритма FastDD и его открытой реализации;
- реализовать алгоритм в рамках Desbordante;
- исследовать возможности улучшения и оптимизации алгоритма и реализовать их;
- реализовать расширение алгоритма с целью поддержки поиска DD согласно более общему определению;
- провести сравнение производительности открытой реализации алгоритма FastDD и реализации данного алгоритма в Desbordante.

2. Обзор

2.1. Дифференциальные зависимости

2.1.1. Определение дифференциальной зависимости, используемое алгоритмом SPLIT

Все определения в данном подразделе повторяют определения из соответствующего подраздела моего отчёта [13] за осень 2023 года и взяты из работы [5]. Рассуждения об эквивалентности определений повторяют секцию обзора моего отчёта [14] за весну 2025 года.

Определение 2.1. Введём следующие обозначения: R — множество всех атрибутов в таблице, $dom(A_i)$ — домен атрибута $A_i \in R$.

Определение 2.2. Метрика d_A — функция, определённая на атрибуте $A \in R$, которая для каждой пары значений $a_1, a_2 \in dom(A)$ возвращает расстояние между ними и удовлетворяет следующим свойствам:

1. $d_A(a_1, a_2) \geq 0$;
2. $d_A(a_1, a_2) = 0 \Leftrightarrow a_1 = a_2$;
3. $d_A(a_1, a_2) = d_A(a_2, a_1)$;
4. $d_A(a_1, a_2) + d_A(a_2, a_3) \geq d_A(a_1, a_3)$,

где $a_1, a_2, a_3 \in dom(A)$.

Определение 2.3. Дифференциальная функция $A[\omega]$ ($\omega = [x, y], 0 \leq x \leq y$) — это функция, определённая на атрибуте A с введённой на нём метрикой d_A и возвращающая булево значение, которое для каждой пары $a_1, a_2 \in dom(A)$ показывает, выполняется ли утверждение $x \leq d_A(a_1, a_2) \leq y$.

Определение 2.4. Дифференциальная функция $X[W_X]$ на множестве атрибутов $X = \{A_1, \ldots, A_m\}$ — это логическое умножение дифференциальных функций для каждого атрибута:

$$X[W_X] = A_1[\omega_1] \wedge A_2[\omega_2] \wedge \ldots \wedge A_m[\omega_m].$$

Говорят, что пара строк таблицы (t, s) удовлетворяет дифференциальной функции $X[W_X]$ ($(t, s) \asymp X[W_X]$), если значение $X[W_X]$ для данной пары строк истинно.

Определение 2.5. Говорят, что дифференциальная функция $X[W_X]$ включает в себя дифференциальную функцию $Y[W_Y]$ ($X[W_X] \succeq Y[W_Y]$), если:

$$\forall A_i[\omega_i] \in X[W_X] \; \exists \; A_i[\omega_i'] \in Y[W_Y] : \omega_i' \subseteq \omega_i.$$

Определение 2.6. Дифференциальная зависимость (DD) — это утверждение $X[W_L] \to Y[W_R]$ между двумя дифференциальными функциями ($X \subset R, Y \subset R$). Говорят, что дифференциальная зависимость удерживается в таблице, если для любой пары строк таблицы (t, s) выполняется следующее утверждение: $(t, s) \asymp X[W_L] \Rightarrow (t, s) \asymp Y[W_R]$.

Определение 2.7. Множество DD Σ_2 является логическим следствием (импликацией) множества DD Σ_1, если для любой таблицы из удерживания всех DD из Σ_1 следует удерживание всех DD из Σ_2.

Определение 2.8. Пусть Σ — множество дифференциальных зависимостей. DD $X[W_L] \to Y[W_R] \in \Sigma$ называется минимальной, если выполняются следующие условия:

1. $\nexists X'[W_X] \to Y[W_R] \in \Sigma : X'[W_X] \succeq X[W_L]$;
2. $\nexists X[W_L] \to Y'[W_Y] \in \Sigma : Y[W_R] \succeq Y'[W_Y]$.

Определение 2.9. Множество DD Σ' называется покрытием для множества DD Σ, если любая DD из Σ лежит в Σ' или является логическим следствием дифференциальных зависимостей из Σ'.

Определение 2.10. Покрытие Σ_c множества Σ называется минимальным, если не существует покрытия $\Sigma' \subset \Sigma_c$.

Задача поиска DD в заданной таблице согласно статье [11] заключается в нахождении минимального покрытия всех удерживающихся в таблице DD, состоящего только из минимальных DD. Приведённый

ниже пример повторяет пример из соответствующего подраздела моего отчёта [13] за осень 2023 года.

Пример 2.1. Пусть $\gamma_1 = X[1,6] \to Y[0,150]$, $\gamma_2 = X[6,7] \to Y[0,100]$, $\gamma_3 = X[1,7] \to Y[0,150]$, $\Sigma = \{\gamma_1, \gamma_2, \gamma_3\}$. $\Sigma_c = \{\gamma_1, \gamma_2\}$ является минимальным покрытием Σ, так как γ_3 логически выводится из $\{\gamma_1, \gamma_2\}$, и, если убрать из Σ_c хотя бы одну из зависимостей, Σ_c перестанет быть покрытием.

Заметим, что γ_1 не является минимальной DD, поскольку она не удовлетворяет условию 1 определения 2.8. Действительно, $\gamma_3 \in \Sigma$ и $X[1,7] \succeq X[1,6]$. Таким образом, если заменить γ_1 на γ_3 в Σ_c, множество $\Sigma'_c = \{\gamma_2, \gamma_3\}$ будет минимальным покрытием Σ, состоящим только из минимальных DD.

Как было показано в моём отчёте [14] за весну 2025 года, оригинальное определение DD, данное в статье [11], можно считать эквивалентным определению, данному в статье [5]. Поскольку определение DD, данное в статье [5], оказалось более простым для реализации, оно использовалось в моих предыдущих работах при реализации алгоритма SPLIT [13, 15].

2.1.2. Определение дифференциальной зависимости, используемое алгоритмом FastDD

Все определения в данном подразделе повторяют определения из соответствующего подраздела моего отчёта [14] за весну 2025 года и взяты из работы [4], предлагающей алгоритм FastDD поиска дифференциальных зависимостей. Некоторые аспекты данных определений, а также рассуждения об эквивалентности были освещены более подробно в соответствующем подразделе моего отчёта [14] за весну 2025 года, в котором было проведено сравнение различных определений дифференциальной зависимости.

Определение 2.11. Дифференциальная функция $\varphi[A]$ — это функция, определённая на атрибуте A с введённой на нём метрикой d_A и возвра-

щающая булево значение, которое для каждой пары $a_1, a_2 \in dom(A)$ показывает, выполняется ли утверждение $d_A(a_1, a_2)$ op x, где $op \in \{\leq, >\}$, x — численное ограничение.

Определение 2.12. Дифференциальная функция $\varphi[X]$ на множестве атрибутов $X = \{A_1, \ldots, A_m\}$ — это логическое умножение дифференциальных функций для каждого атрибута:

$$\varphi[X] = \varphi[A_1] \wedge \varphi[A_2] \wedge \ldots \wedge \varphi[A_m].$$

Говорят, что пара строк таблицы (t, s) удовлетворяет дифференциальной функции $\varphi[X]$ ($(t, s) \asymp \varphi[X]$), если значение $\varphi[X]$ для данной пары строк истинно.

Таблица 1: Пример таблицы (взят из работы [5])

TID	**YoS**	**Gen**	**Edu**	**Sal**
1	25	2	4	15
2	25	1	3	18
3	28	1	3	15
4	32	2	2	12
5	30	1	1	15

Пример 2.2. В качестве примера рассмотрим таблицу 1. Возьмём в качестве метрики для каждой колонки $A \in R$ $d_A(x, y) = |x - y|$. Расстояния между строками 1 и 2 на атрибутах YoS, Gen, Sal равны 0, 1 и 3 соответственно. Таким образом, пара строк $\{1, 2\}$ удовлетворяет дифференциальным функциям $\varphi_1[YoS] = [YoS(\leq 0)], \varphi_2[Gen] = [Gen(\leq 2)], \varphi_3[Sal] = [Sal(> 2)]$. Также ясно, что эта пара строк не удовлетворяет дифференциальным функциям $\varphi_1[YoS] = [YoS(> 0)], \varphi_2[Gen] = [Gen(> 2)], \varphi_3[Sal] = [Sal(\leq 2)]$.

Определение 2.13. Говорят, что дифференциальная функция $\varphi_1[X]$ включает в себя дифференциальную функцию $\varphi_2[X]$ ($\varphi_1[X] \succeq \varphi_2[X]$), если для любой пары строк (t, s) выполняется следующее утверждение: $(t, s) \asymp \varphi_2[X] \Rightarrow (t, s) \asymp \varphi_1[X]$.

Определение 2.14. Дифференциальная зависимость (DD) — это утверждение $\varphi_L[X] \to \varphi_R[A]$ между двумя дифференциальными функциями ($X \subset R, A \in R$). Говорят, что дифференциальная зависимость удерживается в таблице, если для любой пары строк таблицы (t, s) выполняется следующее утверждение: $(t, s) \asymp \varphi_L[X] \Rightarrow (t, s) \asymp \varphi_R[A]$.

Определение 2.15. DD $\gamma = \varphi_L[X] \to \varphi_R[A]$ называется минимальной, если не существует DD $\gamma' = \varphi'_L[Y] \to \varphi'_R[A]$, удерживающейся в таблице, такой что $\varphi'_L[Y] \succeq \varphi_L[X]$ и $\varphi_R[A] \succeq \varphi'_R[A]$.

Задача поиска DD в заданной таблице согласно статье [4] заключается в нахождении всех минимальных DD, удерживающихся в данной таблице.

Пример 2.3. Пусть $\gamma_1 = [X(\leq 6)] \to [Y(\leq 150)]$, $\gamma_2 = [X(> 5)] \to [Y(\leq 100)]$, $\gamma_3 = [X(\leq 7)] \to [Y(\leq 150)]$. Предположим, что $\gamma_1, \gamma_2, \gamma_3$ удерживаются в таблице. Предположим также, что никакие другие DD, кроме вышеуказанных трёх, не удерживаются в таблице. Можно заметить, что γ_1 не является минимальной DD из-за γ_3, так как $[X(\leq 7)] \succeq [X(\leq 6)]$ и $[Y(\leq 150)] \succeq [Y(\leq 150)]$. Также ясно, что γ_2 и γ_3 являются минимальными. Таким образом, $\{\gamma_2, \gamma_3\}$ является множеством всех минимальных DD, удерживающихся в таблице.

Определения дифференциальной функции и дифференциальной зависимости, представленные в статье [4], значительно отличаются от определений, представленных в статье [5]. Согласно новому определению дифференциальная функция в правой части DD определена лишь на одном атрибуте, в то время как в стандартном определении данная дифференциальная функция определена на нескольких атрибутах. Однако данное различие не влияет на эквивалентность определений, поскольку верно следующее утверждение: дифференциальная зависимость $\varphi_L[X] \to \varphi_R[Y]$ ($Y = \{A_1, \ldots, A_n\}$) удерживается тогда и только тогда, когда удерживаются дифференциальные зависимости $\varphi_L[X] \to \varphi_R[A_i]$ $\forall i = 1..n$.

Главное отличие нового определения дифференциальной функции от стандартного состоит в том, что для дифференциальных функций

не определяется их пересечение. Таким образом, для одного атрибута возможны ограничения вида $[A(\leq a)]$ и $[A(> a)]$, но невозможны ограничения вида $[A(= a)]$ и $[A(\geq a, \leq b)]$, в то время как в стандартном определении допускается любой отрезок вида $[a, b]$. Отсюда можно сделать вывод, что класс возможных дифференциальных функций становится более узким, поэтому новое определение дифференциальной функции, а следовательно, и дифференциальной зависимости, нельзя считать эквивалентным стандартному. Невозможность использования произвольных отрезков в качестве ограничений на расстояние делает определение DD, данное в статье [4], более узким, чем определения, данные в статьях [11, 5]. Более подробно о сравнении трёх вариантов определения DD написано в моём отчёте [14] за весну 2025 года.

2.1.3. Выводы

В данном подразделе представлены выводы, касающиеся главных отличий в использовании понятия дифференциальной зависимости алгоритмами SPLIT и FastDD, представленных в предыдущих подразделах.

Главным отличием, как было показано ранее, является тот факт, что определение DD, используемое алгоритмом FastDD, является более узким, чем определение DD, используемое алгоритмом SPLIT. Таким образом возникла необходимость расширения реализации алгоритма FastDD для поддержки более общего определения DD и поиска более широкого класса зависимостей.

Ещё одним отличием между алгоритмами SPLIT и FastDD является решаемая ими задача. Алгоритмом SPLIT решается задача поиска минимального покрытия всех удерживающихся в таблице DD, состоящего только из минимальных зависимостей. Алгоритм FastDD формально решает немного другую задачу: он ищет все минимальные DD, удерживающиеся в таблице. Однако на практике, как будет показано в разделе описания решения, данное различие является несущественным, так как реализации обоих алгоритмов возвращают потенциально не полностью минимальное покрытие.

2.2. Алгоритм FastDD

Алгоритм FastDD поиска дифференциальных зависимостей был предложен в 2024 году в статье [4]. Подробный обзор алгоритма был представлен в моём отчёте [14] за весну 2025 года. В данном подразделе будет проведён краткий обзор алгоритма со ссылками на вышеуказанный отчёт.

2.2.1. Определения, используемые в алгоритме FastDD

В данном подразделе вводятся определения, использующиеся в описании алгоритма. Все определения и утверждения в данном подразделе повторяют определения и утверждения из соответствующего подраздела моего отчёта [14] за весну 2025 года.

Определение 2.16. Обозначим за Ψ некоторое множество, состоящее из дифференциальных функций $\varphi[A], A \in R$. Множество различий для пары строк (t, s) — это множество следующего вида: $D(t, s) = \{\varphi[A] \in \Psi | (t, s) \not\asymp \varphi[A]\}$.

Определение 2.17. Множество различий для экземпляра отношения r — это множество следующего вида: $D_r = \{D(t, s) | t \in r, s \in r, D(t, s) \neq \emptyset\}$.

Определение 2.18. Пусть $\varphi[A] \in \Psi$. Обозначим за $D_r(\varphi[A])$ множество множеств различий, содержащих в себе $\varphi[A]$: $D_r(\varphi[A]) = \{U | U \in D_r \wedge \varphi[A] \in U\}$.

Утверждение 2.1. DD $\varphi_L[X] \rightarrow \varphi[A]$, где $\varphi[X] = \bigwedge_{A_i \in X} \varphi_i[A_i], A \notin X$, удерживается в таблице (то есть экземпляре отношения r) тогда и только тогда, когда $\forall U \in D_r(\varphi[A])\ \exists \varphi_i[A_i] : \varphi_i[A_i] \in U$.

Утверждение 2.1 связывает задачу поиска дифференциальных зависимостей с задачей поиска вершинного покрытия в гиперграфе. Следующие определения, связанные с понятиями гиперграфа и его вершинного покрытия, взяты из работ [7] и [10] и повторяют секцию обзора моего отчёта [12] за весну 2024 года.

Определение 2.19. Гиперграф $\mathcal{F}$ — множество подмножеств множества вершин V ($F = \{F_1, \ldots, F_m\}, F_i \subseteq V$). F_i — рёбра гиперграфа. $|F|$ — число рёбер гиперграфа.

Определение 2.20. Вершинное покрытие гиперграфа $\mathcal{F}$ (transversal, hitting set) — подмножество V, имеющее непустое пересечение с каждым ребром $\mathcal{F}$.

Определение 2.21. Вершинное покрытие S гиперграфа $\mathcal{F}$ называется минимальным, если не существует вершинного покрытия $S_1 : S_1 \subset S$.

Утверждение 2.1 позволяет сделать вывод, что задача поиска удерживающихся в таблице DD с правой частью $\varphi[A]$ равносильна задаче поиска вершинного покрытия гиперграфа $D_r(\varphi[A])$. Однако задача нахождения всех минимальных DD, удерживающихся в таблице, не является равносильной задаче поиска минимального вершинного покрытия гиперграфа, поскольку, кроме минимальности по включению в теоретико-множественном смысле, для дифференциальных зависимостей также требуется минимальность по включению в себя (2.13). Для правильности результатов поиска зависимостей требуются дополнительные проверки на их минимальность, поэтому необходимо изменение стандартного алгоритма поиска минимального вершинного покрытия. Более подробно изменённый алгоритм описан в моём отчёте [14] за весну 2025 года.

2.2.2. Описание алгоритма

В данном подразделе представлен краткий обзор алгоритма FastDD, повторяющий соответствующий подраздел моего отчёта [14] за весну 2025 года. Алгоритм FastDD поиска дифференциальных зависимостей состоит из трёх основных стадий:

1. Препроцессинг
2. Построение множества различий
3. Поиск и минимизация вершинных покрытий

Схема работы алгоритма FastDD показана на рисунке 1. На этапе препроцессинга алгоритмом считывается входная таблица, а также происходит построение пространства поиска дифференциальных зависимостей. На этапе построения множества различий с использованием созданного ранее пространства поиска строится множество различий. На этапе поиска и минимизации вершинных покрытий для полученного на предыдущей стадии множества различий строятся вершинные покрытия, которые состоят из удерживающихся в таблице DD, после чего из полученных зависимостей удаляются DD, не являющиеся минимальными. В результате работы алгоритмом выдаются все минимальные дифференциальные зависимости, удерживающиеся в таблице. Более подробно каждая стадия алгоритма описана в соответствующих подразделах моего отчёта [14] за весну 2025 года.

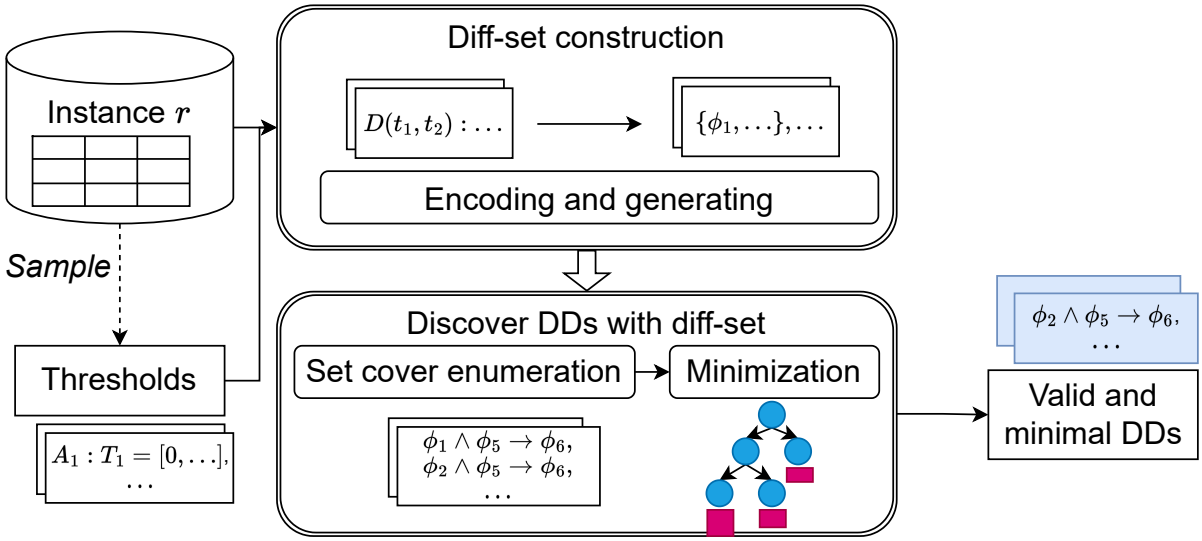


Рис. 1: Схема алгоритма FastDD (взято из работы [4])

2.3. Открытая реализация алгоритма

Открытая реализация алгоритма FastDD, написанная авторами статьи [4] на языке программирования Java, доступна на GitHub[7]. В данном подразделе представлено описание данной реализации. Некоторые

[7] https://github.com/TristonK/FastDD

детали реализации повторяют подробное описание алгоритма, представленное в моём отчёте [14] за весну 2025 года.

2.3.1. Препроцессинг

Главный класс реализации алгоритма под названием `FastDD` содержит два поля: поле класса `Input`, отвечающего за хранение входной таблицы, и поле класса `DifferentialFunctionBuilder`, отвечающего за построение и хранение пространства поиска дифференциальных зависимостей. На этапе препроцессинга алгоритмом считывается входная таблица и строится пространство поиска. В работе [4] описаны два основных подхода к его заданию: автоматическое определение ограничений, являющихся составными частями дифференциальных функций из пространства поиска, а также предоставление пользовательских ограничений. Открытая реализация алгоритма позволяет использовать оба подхода к построению пространства поиска.

Пользователь имеет возможность указать ограничения в отдельном текстовом файле в специальном формате. Пример такого файла представлен на рисунке 2. Для каждой колонки в первом списке перечисляются ограничения вида $\leq$, а во втором — ограничения вида $>$. Каждое значение в списке определяет дифференциальную функцию, попадающую в пространство поиска. Сами значения в списках, а также их количество определяются пользователем.

Если же пользователь не указывает путь к файлу в качестве параметра алгоритма, то ограничения вычисляются автоматически: из таблицы случайным образом выбирается 200 записей, между которыми вычисляются расстояния на каждой колонке. Из получившегося множества расстояний несколько значений выбираются в качестве ограничений на основе частоты, с которой данные значения встречаются. На основе получившихся ограничений класс `DifferentialFunctionBuilder` строит пространство поиска.

Col0 [3,10,1][2,7,1]
Col1 [2,5,4][3,0,0]
Col2 [2,8][3,5,4,0]
Col3 [4,0,3][1,4]
Col4 [1,6,4][0,4]
Col5 [3,9][1,2,7]

Рис. 2: Пример текстового файла пользовательских ограничений

2.3.2. Построение множества различий

Открытая реализация алгоритма FastDD использует несколько техник для увеличения производительности стадии построения множества различий D_r. Например, все нужные расстояния подсчитываются между каждой парой кластеров равных значений, количество которых обычно существенно меньше, чем число всех пар строк. Для экономного хранения всех кластеров используются специальные структуры данных — индексы списков позиций (Position List Index, PLI) [6]. Помимо этого, подсчёт расстояний оптимизируется для атрибутов, значения которых могут быть отсортированы. Для больших датасетов используется техника шардирования: таблица разбивается на несколько блоков, для каждого из которых вычисляется своё множество различий, после чего все полученные множества объединяются между собой. С помощью этого подхода достигается ускорение подсчёта расстояний. Кроме того, такая техника допускает параллельное выполнение для разных блоков, что помогает достичь существенного ускорения на процессорах, состоящих из нескольких ядер.

В реализации алгоритма класс `PliShard` отвечает за индексы списков позиций и их шардирование, а класс `PliShardBuilder` — за их построение. За построение множеств различий отвечает класс под названием `DFSetBuilder`, строящий специальную сокращённую запись множеств различий для каждой пары строк с целью экономии потребляемой памяти. Для своей работы `DFSetBuilder` использует объекты классов `DifferentialFunctionBuilder` и `PliShard`. За сокращение записи отвечает базовый класс `ISNBuilder`, а также классы-наследники `SingleISNBuilder` и `CrossISNBuilder`, вычисляющие сокращённую за-

пись для пар строк из одного блока и из различных блоков соответственно. Результатом работы `DFBuilder` является объект класса `DFSet`. Опуская детали, можно сказать, что данный объект хранит в себе множество битсетов, для которых каждая единица означает то, что пара строк, соответствующая данному битсету, удовлетворяет дифференциальной функции с соответствующим номером из пространства поиска. Данное множество и является множеством различий D_r.

2.3.3. Поиск и минимизация вершинных покрытий

За выполнение данной стадии алгоритма в его реализации отвечает класс `HybridEvidenceInversion`, использующий для своей работы построенный на предыдущей стадии объект класса `DFSet`. В процессе работы для каждой дифференциальной функции $\varphi[A]$ из пространства поиска, выступающей в качестве правой части дифференциальных зависимостей (доступ к пространству поиска дифференциальных функций осуществляется с помощью `DifferentialFunctionBuilder`), строится гиперграф $D_r(\varphi[A])$ на основе множества различий. За поиск вершинного покрытия в каждом таком гиперграфе отвечает класс под названием `EvidenceInversion`. После получения вершинных покрытий происходит их минимизация, полученное множество DD с правой частью $\varphi[A]$ добавляется к окончательному результату алгоритма.

При поиске и минимизации вершинных покрытий используются специальные деревья, хранящие битсеты и позволяющие эффективно решать задачу поиска битсетов, являющихся подмножествами и надмножествами заданного. На данной стадии используются три различные модификации деревьев вышеуказанного вида, реализованные в классах `TranslatingTreeSearch`, `TreeSearch` и `TranslatingMinimizeTree`. Благодаря эффективности данных деревьев стадия поиска и минимизации вершинных покрытий является наиболее производительной, обеспечивая превосходство алгоритма FastDD над алгоритмом SPLIT по скорости работы.

3. Описание решения

В данном разделе представлены подробности реализации алгоритма FastDD в рамках Desbordante. За основу была взята открытая реализация[8] алгоритма от авторов статьи [4].

3.1. Первоначальная реализация

3.1.1. Базовые классы и структуры

Любой алгоритм в Desbordante обязан иметь собственный класс, являющийся наследником базового класса `Algorithm`. В связи с тем, что в рамках Desbordante уже был реализован алгоритм SPLIT поиска дифференциальных зависимостей, было принято решение создать промежуточный класс `DDAlgorithm`, являющийся наследником класса `Algorithm` и отвечающий за общую для всех алгоритмов поиска DD функциональность. Класс `DDAlgorithm` хранит коллекцию DD, получаемых алгоритмами в результате их работы. Классы алгоритмов поиска DD (`Split` и `FastDD`) были сделаны наследниками класса `DDAlgorithm`.

Листинг 1: Базовые структуры

```
enum class Operator { kLessOrEqual, kGreater };

class DifferentialFunction {
    Operator operator_;
    double threshold_;
    Column const* column_;
};

class DifferentialDependency {
    std::vector<DifferentialFunction> lhs_;
    DifferentialFunction rhs_;
};
```

[8] https://github.com/TristonK/FastDD

Для внутреннего представления DD были созданы базовые структуры `DifferentialFunction` и `DifferentialDependency`, реализация которых представлена в листинге 1. Структура `DifferentialFunction` хранит в себе оператор ($\leq$ или $>$), ограничение и указатель на колонку, на которую накладывается данное ограничение (колонка представляется уже реализованным в Desbordante классом `Column`). Дифференциальная зависимость $\varphi_L[X] \rightarrow \varphi[A]$, где $\varphi[X] = \bigwedge_{A_i \in X} \varphi_i[A_i], A \notin X$, представляется набором объектов `DifferentialFunction` в левой части, соответствующих дифференциальным функциям $\varphi_i[A_i]$, а также одним объектом класса `DifferentialFunction`, соответствующим правой части DD.

Стоит отметить, что из-за различия определений DD [14], используемых алгоритмами SPLIT и FastDD, результаты алгоритмов имеют разные типы. Для унификации возвращаемых значений, хранящихся в `DDAlgorithm`, в первоначальной реализации было решено выполнять приведение объектов класса `DifferentialDependency`, возвращаемых алгоритмом FastDD, к внешнему представлению DD, используемому в Desbordante. Как показано в разделе 3.2, алгоритм FastDD был модифицирован с целью поддержки более общего определения DD, что позволило окончательно унифицировать представления DD в Desbordante.

В следующих подразделах будут представлены особенности реализации стадий алгоритма FastDD в рамках Desbordante.

3.1.2. Препроцессинг

Для чтения входной таблицы в Desbordante используется реализованный там класс `InputTable`, а для хранения полученной информации — класс `ColumnLayoutTypedRelationData`, позволяющий также узнать типы данных атрибутов, хранящихся в таблице. Информация о типах данных помогает подобрать нужную метрику для подсчёта расстояний на каждом атрибуте. Для подсчёта расстояний было решено использовать уже реализованные в Desbordante стандартные метрики для каждого типа, для вычисления которых используется метод `Dist` интерфейса `IMetrizableType`, переопределяемый типами, поддержива-

емыми в Desbordante.

Задание пространства поиска было реализовано в соответствии с открытой реализацией алгоритма, за исключением формата ввода пользовательских ограничений. В реализации алгоритма в Desbordante был использован табличный формат ввода, в то время как в открытой реализации был использован текстовый. В таблице 2 представлен пример файла пользовательских ограничений, соответствующий примеру для открытой реализации, представленному на рисунке 2.

Таблица 2: Пример таблицы пользовательских ограничений

Col0	**Col1**	**Col2**	**Col3**	**Col4**	**Col5**
$<= 3$	$<= 2$	> 3	$<= 4$	$<= 1$	> 1
> 2	> 3	$<= 2$	> 1	> 0	$<= 3$
$<= 10$	> 0	> 5	> 4	$<= 6$	> 2
> 7	$<= 5$	> 4	$<= 0$	> 4	> 7
> 1	$<= 4$	> 0	$<= 3$	$<= 4$	$<= 9$
$<= 1$	> 0	$<= 8$	—	—	—

3.1.3. Построение множества различий

Реализация данной стадии алгоритма в Desbordante соответствует открытой реализации. Для данной стадии были созданы следующие классы: `DifferentialFunctionBuilder`, `PliShard`, `SingleISNBuilder`, `CrossISNBuilder`, `DiffSetBuilder` (соответствует `DFSetBuilder` из открытой реализации) и `DiffSet` (соответствует `DFSet`). Встроенная в Desbordante система типов позволила обобщить оптимизацию для атрибутов, значения которых могут быть отсортированы. В то время как реализация авторов статьи [4] поддерживает только целочисленные значения, значения с плавающей точкой и строки, реализация алгоритма в Desbordante поддерживает любые типы, для которых может быть вычислено расстояние, например тип данных «Дата». Поскольку вышеуказанный тип данных можно отсортировать, для него также применима оптимизация построения множества различий. Таким образом, реализация FastDD в Desbordante эффективно поддерживает большее

количество типов данных и является расширяемой, позволяя без дополнительных усилий добавлять поддержку новых типов данных.

3.1.4. Поиск и минимизация вершинных покрытий

Реализация данной стадии алгоритма в Desbordante также соответствует открытой реализации. Для данной стадии были реализованы следующие классы: `HybridEvidenceInverter` (соответствует классу `HybridEvidenceInversion`), `EvidenceInverter` (соответствует классу `EvidenceInversion`), а также классы, реализующие деревья поиска: `TranslatingTreeSearch`, `TreeSearch` и `TranslatingMinimizeTree`. Единственным отличием реализации данной стадии в Desbordante является вышеописанный перевод полученных дифференциальных зависимостей в другое представление с целью его унификации.

3.1.5. Диаграмма классов

На рисунке 3 представлена UML-диаграмма классов, описывающая главные компоненты реализации алгоритма FastDD в Desbordante.

3.2. Оптимизации и улучшения алгоритма

3.2.1. Расширение алгоритма

В статье [4] было использовано более узкое определение дифференциальной зависимости, что обсуждалось в разделе 2.1 и что было подробно разобрано в моём отчёте [14] за весну 2025 года. Таким образом, алгоритмом FastDD находятся только DD с ограничениями на расстояние вида $\leq$ и $>$, в то время как реализация алгоритма SPLIT в Desbordante ищет DD согласно оригинальному определению, допускающему ограничения на расстояние в виде любых отрезков. В связи с этим было решено расширить реализацию алгоритма FastDD для поддержки более общего определения DD.

Базовые структуры были переработаны с использованием уже существующей структуры `DFConstraint`, хранящей нижнюю и верхнюю

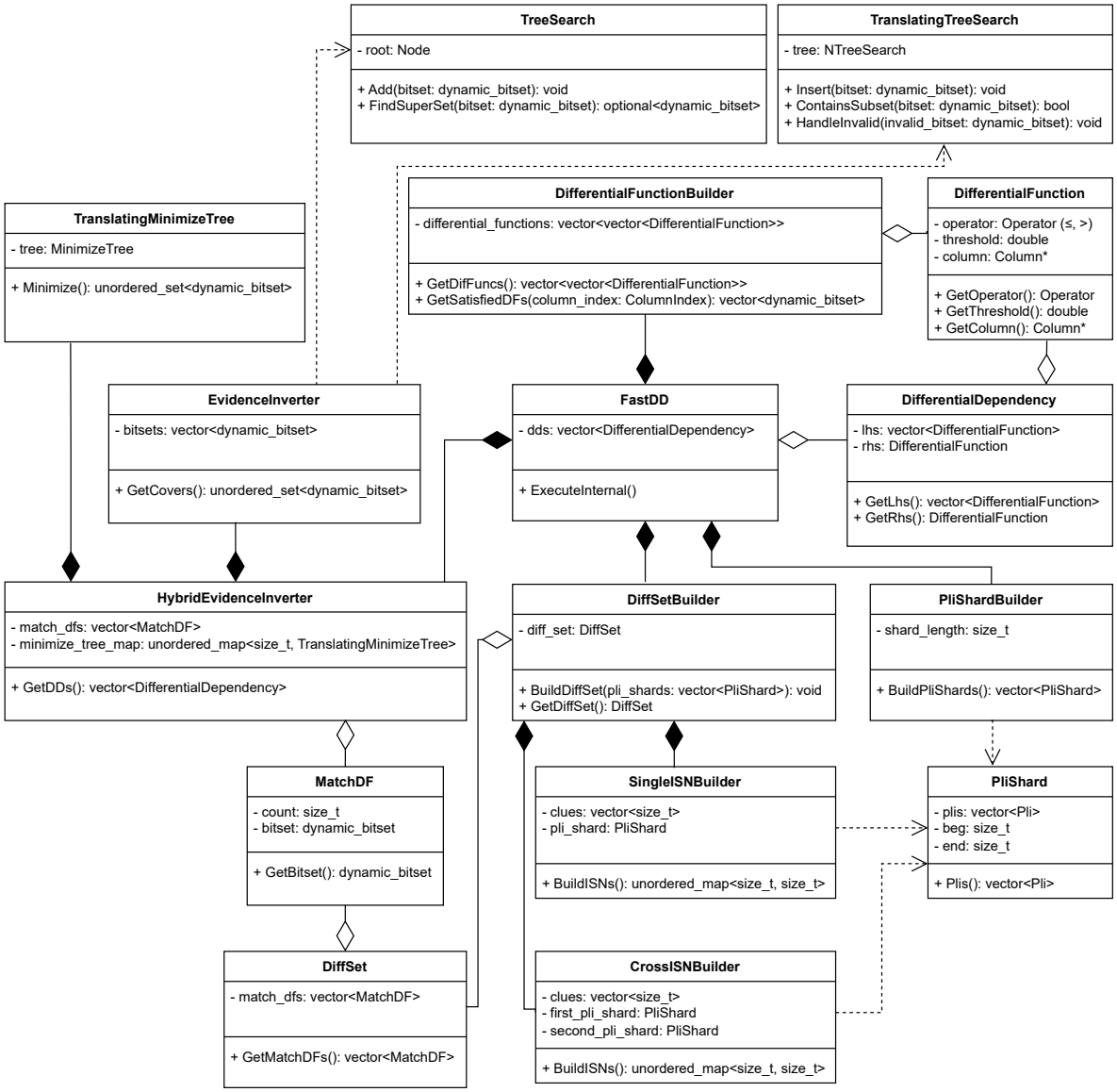


Рис. 3: UML-диаграмма классов алгоритма FastDD

границы отрезка допустимых расстояний.

Листинг 2: Переработанные базовые структуры

```
struct DFConstraint {
    double lower_bound_;
    double upper_bound_;
};

class DifferentialFunction {
    DFConstraint constraint_;
    Column const* column_;
};

class DifferentialDependency {
    std::vector<DifferentialFunction> lhs_;
    DifferentialFunction rhs_;
};
```

Для этапа препроцессинга был изменён формат пользовательского ввода пространства поиска: в таблице пользовательских ограничений стало требоваться указание отрезков допустимых расстояний для каждого атрибута. Данный формат был использован в реализации алгоритма SPLIT в Desbordante. В таблице 3 представлен пример файла пользовательских ограничений для более общего определения DD, соответствующий примеру для первоначальной реализации, представленному в таблице 2. Стоит отметить, что для представления ограничений вида $[A(> a)]$ в терминах стандартного определения в таблице необходимо использовать конструкцию вида $[a + 0.001; inf]$.

Кроме того, для поддержки более общего определения появилась необходимость в изменении стадии 2 алгоритма (построения множества различий). Алгоритм представляет множество различий для каждой пары строк в компактной числовой форме, по которой можно однозначно определить, каким дифференциальным функциям удовлетворяет заданная пара строк. Приведённое ниже подробное описание стадии

Таблица 3: Пример таблицы пользовательских ограничений

Col0	**Col1**	**Col2**	**Col3**	**Col4**	**Col5**
[0;3]	[0;2]	[3.001;inf]	[0;4]	[0;1]	[1.001;inf]
[2.001;inf]	[3.001;inf]	[0;2]	[1.001;inf]	[0.001;inf]	[0;3]
[0;10]	[0.001;inf]	[5.001;inf]	[4.001;inf]	[0;6]	[2.001;inf]
[7.001;inf]	[0;5]	[4.001;inf]	[0;0]	[4.001;inf]	[7.001;inf]
[1.001;inf]	[0;4]	[0.001;inf]	[0;3]	[0;4]	[0;9]
[0;1]	[0.001;inf]	[0;8]	—	—	—

было взято из моего отчёта [14] за весну 2025 года.

Предположим, что для каждого атрибута $A_i \in R$ определено множество численных ограничений T_i, которое было отсортировано по возрастанию. Эти ограничения порождают следующие отрезки разбиения: $(T_i[0], T_i[1]], \ldots, (T_i[|T_i| - 1], \infty)$. Расстояние между любой парой строк из таблицы на атрибуте A_i попадает в один из $|T_i|$ отрезков разбиения, представленных выше. Таким образом, любому расстоянию d однозначно сопоставляется номер отрезка разбиения $\#_{A_i}(d)$, в который попадает это расстояние. Данный номер принимает значения от 0 до $|T_i|$.

Множество различий $D(t, s)$ однозначно представляется в компактной числовой форме с помощью следующей формулы: $D(t, s) = a_1 + a_2 * S_1 + \cdots + a_{|R|} * S_{|R|-1}$, где $a_i = \#_{A_i}(d_{A_i}(t_{A_i}, s_{A_i})), S_i = \prod_{k=1}^{i}(|T_k| + 1)$. По номеру, который получается с помощью данной формулы, можно однозначно определить значения a_i, из которых с использованием известных T_i можно однозначно определить, удовлетворяет ли пара строк (t, s) произвольной дифференциальной функции $\varphi_i[A_i]$.

Данный метод основывается на том, что номер отрезка однозначно определяет набор дифференциальных функций на атрибуте, которым удовлетворяет заданная пара строк.

Пример 3.1. Допустим, что для атрибуте A определены следующие дифференциальные функции: $\varphi_1 = [A(\leq 5)], \varphi_2 = [A(\leq 9)], \varphi_3 = [A(> 7)]$. Расстояние между любой парой неравных значений попадает в один из четырёх отрезков разбиения: $(0, 5], (5, 7], (7, 9], (9, \infty)$. Если расстояние $d_A(t, s) \in (0, 5]$, то (t, s) удовлетворяет φ_1, φ_2. Если $d_A(t, s) \in (5, 7]$,

то (t, s) удовлетворяет φ_2. Если $d_A(t, s) \in (7, 9]$, то (t, s) удовлетворяет φ_2, φ_3. Если $d_A(t, s) \in (9, \infty)$, то (t, s) удовлетворяет φ_3. Таким образом, каждый номер отрезка определяет уникальный набор дифференциальных функций, что позволяет установить взаимно однозначное соответствие между множеством различий и его компактной формой. На рисунке 4 показаны отрезки разбиения и их номера.

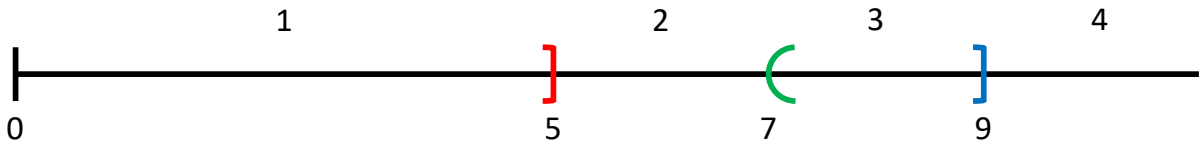


Рис. 4: Номера отрезков разбиения

В случае расширенного определения DD взаимно однозначного соответствия может не быть: разные отрезки разбиения могут соответствовать одинаковому набору дифференциальных функций. Для построения компактной формы множества различий все отрезки, соответствующие одинаковым наборам дифференциальных функций, были объединены в зоны. Каждая зона соответствует уникальному набору дифференциальных функций. Таким образом, для установления взаимно однозначного соответствия используется не номер отрезка, а номер зоны.

Пример 3.2. Допустим, что для атрибуте A определены следующие дифференциальные функции: $\varphi_1 = A[0, 8], \varphi_2 = A[3, 5], \varphi_3 = A[7, 9]$. Расстояние между любой парой неравных значений попадает в один из шести отрезков разбиения: $[0, 3), [3, 5], (5, 7), [7, 8], (8, 9], (9, \infty)$. Наборы дифференциальных функций, соответствующих данным отрезкам, таковы:

1. $[0, 3) : \{\varphi_1\}$,
2. $[3, 5] : \{\varphi_1, \varphi_2\}$,
3. $(5, 7) : \{\varphi_1\}$,
4. $[7, 8] : \{\varphi_1, \varphi_3\}$,

5. $(8, 9] : \{\varphi_3\}$,

6. $(9, \infty) : \{\}$.

Таким образом, для шести отрезков разбиения строятся всего пять зон, так как отрезки $[0, 3)$ и $(5, 7)$ принадлежат одной зоне, соответствующей набору дифференциальных функций $\{\varphi_1\}$. На рисунке 5 показаны отрезки разбиения и соответствующие им номера зон.

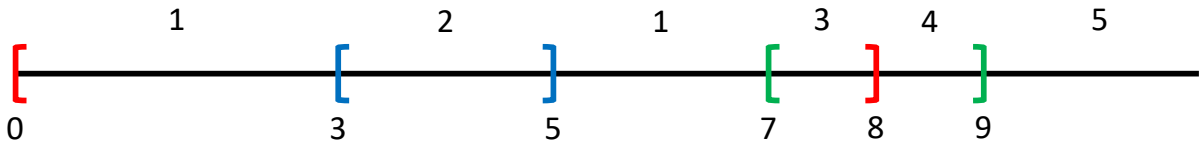


Рис. 5: Номера зон

3.2.2. Улучшение выдачи алгоритма

Зачастую найденные алгоритмом FastDD дифференциальные зависимости могут быть неинтересны пользователю из-за их тривиальности. Существуют три случая, когда DD можно убрать из результата:

1. Правая или левая часть зависимости выполняются для любой пары строк (DD тривиальна).

2. Левая часть зависимости не выполняется ни для одной пары строк. В этом случае DD формально выполняется, но является нерепрезентативной, оказываясь бесполезной для пользователя.

3. DD выводится из других зависимостей по транзитивности. Если в результате алгоритма были найдены DD $A \to B$ и $B \to C$, то DD $A \to C$ можно не выводить, так как она является прямым следствием из первых двух DD. Более формально о правилах вывода одних DD из других, а также о понятии минимального покрытия описано в оригинальной статье [11]. Стоит отметить, что алгоритмы SPLIT и FastDD не удаляют транзитивно выводимые DD из результата, потенциально выдавая не полностью минимальное покрытие.

Для решения этих задач в алгоритм были добавлены небольшие дополнения. В начале работы алгоритма был добавлен подсчёт минимального и максимального расстояния между значениями для каждого атрибута. Данные значения затем пересекаются с дифференциальными функциями из пространства поиска для обеспечения отсутствия дифференциальных функций, которым не удовлетворяет ни одна пара строк, а также дифференциальных функций, которым удовлетворяют все пары строк.

Кроме того, в конец алгоритма была добавлена фильтрация найденных зависимостей. В процессе фильтрации проверяется, удовлетворяет ли левой части хотя бы одна пара строк, а также является ли DD выводимой из других DD по транзитивности. В случае, если DD не является транзитивно выводимой, и её левой части удовлетворяет хотя бы одна пара строк, то такая зависимость добавляется в окончательный результат. Из-за большой сложности поиска транзитивно выводимых зависимостей в рамках реализации был сделан лишь поиск DD, выводимых напрямую из двух других. Поскольку данные проверки являются вычислительно дорогими, в случае большого числа найденных DD время выполнения алгоритма может значительно увеличиться. В связи с этим возможно добавление пользовательской опции алгоритма, позволяющей отключить фильтрацию.

3.2.3. Новая стратегия поиска и минимизации вершинных покрытий

Как было показано в обзоре, алгоритм FastDD сводит задачу поиска всех минимальных DD, удерживающихся в таблице, к задаче поиска минимального вершинного покрытия в гиперграфе. Авторы статьи [4] предлагают собственный алгоритм для данной задачи с добавлением последующих проверок зависимостей на минимальность. Однако существуют и другие алгоритмы поиска минимального вершинного покрытия, описанные в статье [10]. Один из наиболее производительных алгоритмов под названием MMCS использовался для алгоритма HPIValid поиска уникальных комбинаций колонок (UCC) [7], где была подтвер-

ждена его высокая производительность. Таким образом, было решено реализовать новую стратегию поиска минимального вершинного покрытия на основе MMCS, модифицированную под дифференциальные зависимости, для сравнения с уже существующей стратегией.

За основу была взята реализация MMCS из алгоритма HPIValid, интегрированного в Desbordante [12]. Для учёта специфики дифференциальных зависимостей было сделано следующее изменение: в процессе добавления новых вершин в вершинное покрытие было гарантировано, что для каждого атрибута может быть добавлена максимум одна вершина. Такая проверка нужна для того, чтобы в результате не оказалось зависимостей, в левой части которых присутствует более одной дифференциальной функции на одном атрибуте. Проверки на минимальность зависимостей, производящиеся для полученного минимального вершинного покрытия, остались без изменений.

3.2.4. Использование битсетов с фиксированным размером

В реализации алгоритма FastDD для поиска и минимизации вершинных покрытий активно используются битсеты. Как показано в обзоре открытой реализации алгоритма, множество различий, передающееся на вход данной стадии алгоритма, представляется в памяти множеством битсетов, размер которых совпадает с размером пространства поиска, т.е. с количеством дифференциальных функций, среди которых ищутся зависимости. Также на данной стадии используются специальные деревья, предназначенные для эффективного поиска битсетов. Для стратегии, использующей алгоритм MMCS, также применяются битсеты. Их использование делает алгоритм FastDD производительным. Однако размер пространства поиска известен только во время выполнения алгоритма, что вынуждает использовать реализации битсетов с динамическим размером. В первоначальной реализации алгоритма в Desbordante используется реализация битсета с динамическим размером из библиотеки Boost: `boost::dynamic_bitset`.

Битсеты с динамическим размером требуют динамического выделения памяти, что делает их менее производительными, чем битсеты

с фиксированным размером, известным на этапе компиляции. Таким образом, использование битсетов с фиксированным размером может улучшить производительность алгоритма. Однако битсеты с фиксированным размером не могут покрыть все случаи, так как размер пространства поиска зависит от пользователя и может быть неограниченным. Поэтому было решено использовать битсеты с фиксированным размером только для небольших пространств поиска, размер которых не превышает 128. Данный размер покрывает случаи, когда пользователю не требуется большое число дифференциальных функций для поиска DD.

Для реализации алгоритма с использованием битсетов с фиксированным размером был сделан класс-обёртка над реализацией битсета из стандартной библиотеки: `std::bitset<N>`, где N — константа. Данная обёртка получила название `StaticBitset<N>`, и данный класс, помимо самого битсета, хранит его реальный размер для правильной индексации. Кроме того, все классы алгоритма, использующие битсеты в своей реализации, были сделаны шаблонами, принимающими тип битсета. Если размер пространства поиска, заданного пользователем, не больше 32, то алгоритм использует `StaticBitset<32>`. Если размер находится в промежутке от 32 до 64, то используется `StaticBitset<64>`. Если размер находится в промежутке от 64 до 128, то используется `StaticBitset<128>`. Если же размер пространства поиска больше 128, то используется `boost::dynamic_bitset`. Таким образом достигается улучшенная производительность алгоритма на небольших пространствах поиска.

3.2.5. Технические оптимизации

Для ускорения алгоритма также были проведены оптимизации некоторых структур данных. Основная их часть коснулась класса `TranslatingTreeSearch`, выполняющего основную работу на стадии поиска и минимизации вершинных покрытий. Этот класс представляет собой специальную реализацию префиксного дерева для поиска битсетов. В каждом узле этого дерева хранится структура `std::unordered_map`,

сопоставляющая индексу в битсете указатель на ребёнка данной вершины, хранящего битсеты с установленной 1 в данном индексе. Сами битсеты хранятся лишь в листьях дерева.

Для ускорения поиска вершин-потомков из заданной вершины было решено заменить структуру `std::unordered_map` на `std::vector`, хранящий для каждого возможного индекса потенциального ребёнка (если ребёнка нет, то хранится значение `nullptr`). В случае, если число потомков вершины небольшое, то в `std::vector` хранится много лишней информации, что увеличивает потребление памяти. Однако на практике значительного увеличения потребляемой памяти удалось избежать с помощью ещё одной оптимизации — ленивого создания вершин.

В первоначальной реализации дерева при добавлении битсета требовалось для каждой установленной 1 перейти к ребёнку по соответствующему индексу. Если же вершины-ребёнка ещё не было создано, то она создавалась. При ленивом создании при первоначальном добавлении битсета все эти вершины не создаются, и битсет кладётся в первый свободный лист на пути из корня. Только в том случае, если при добавлении нового битсета в листе дерева уже есть битсет, данный лист расщепляется, и создаются две новые вершины-ребёнка, в которые кладутся два битсета. Таким образом уменьшается число созданных вершин, что экономит как память, так и время поиска битсета в дереве.

Среди других небольших оптимизаций можно выделить замену структуры `std::unordered_set`, активно использующейся для определения уникальных компактных числовых форм записи множеств различий, на более быструю альтернативу `boost::unordered::unordered_flat_set`. Также для оптимизации применялось точечное переписывание наиболее затратных методов.

Кроме того, ускорению стадии построения множеств различий способствовал подбор оптимального размера блока, на которые разбивается исходная таблица. Слишком большой размер блока вызывает увеличение потребляемой памяти, а также времени на вычисление компактной формы множеств различий. Слишком маленький размер блока также ведёт к увеличению времени работы из-за накладных расходов по

объединению результатов, полученных в каждом блоке. В результате экспериментов был определён оптимальный размер блока — 800 строк.

Таблица 4: Датасеты

Датасет	#колонок	#строк	#DF	#DD
adult	15	32561	75	987
atom	13	50000	65	1913
claim	11	20000	55	252
digits	15	1797	75	210931
fd_reduced	20	10000	100	101589
flight	13	20000	65	1365
letter	14	1000	70	569142
neighbors	7	100000	35	14
spstock	7	50000	35	1109
tax	15	12106	75	12202

4. Эксперимент

Для сравнения производительности различных реализаций алгоритма FastDD были использованы его открытая реализация[9] [4], а также первоначальная и оптимизированная реализации алгоритма в Desbordante. Для тестирования использовалась однопоточная версия открытой реализации алгоритма. Эксперименты проводились на ноутбуке с установленной операционной системой Linux Ubuntu 24.04, 8 ГБ оперативной памяти и 8 процессорами Intel(R) Core(TM) i5-9300H CPU @ 2.40GHz 2.40 GHz (максимальная частота — 4.1 GHz). При проведении экспериментов был выключен файл подкачки, сброшены кэши и зафиксирована максимальная частота процессоров. Все представленные показатели времени работы алгоритмов являются усреднёнными значениями по пяти замерам. Для формирования пространства поиска для всех таблиц для каждого атрибута использовалось по 5 дифференциальных функций. Все реализации алгоритма запускались с размером блока, равным 800 строк.

В таблице 4 представлены датасеты, на которых тестировалась производительность различных версий алгоритма FastDD. #DF означает общее число дифференциальных функций, использованных для построения пространства поиска. #DD означает число дифференциальных

[9] https://github.com/TristonK/FastDD

Таблица 5: Время работы (с)

Датасет	Откр.	Перв.	Опт.	Откр./Опт.
adult	209.904	272.346	**24.245**	8.66×
atom	49.967	26.084	**17.093**	2.92×
claim	72.641	45.760	**23.290**	3.12×
digits	159.612	191.789	**11.608**	13.75×
fd_reduced	238.616	308.494	**42.487**	5.62×
flight	48.638	28.517	**17.657**	2.75×
letter	91.911	144.987	**7.818**	11.76×
neighbors	287.691	101.970	**68.879**	4.18×
spstock	210.093	38.514	**33.675**	6.24×
tax	74.126	59.367	**25.407**	2.92×

зависимостей, найденных алгоритмом.

4.1. Время работы и потребляемая память

В таблице 5 представлено время работы открытой, первоначальной и оптимизированной реализаций алгоритма FastDD на различных датасетах. Колонка 'Откр.' показывает время работы открытой реализации; колонка 'Перв.' показывает время работы первоначальной реализации в Desbordante; колонка 'Опт.' показывает время работы оптимизированной реализации в Desbordante.

Из таблицы можно заметить, что оптимизированная реализация алгоритма FastDD в Desbordante превосходит по производительности как первоначальную, так и открытую реализацию на всех датасетах. Однако значения ускорения на разных датасетах отличаются, варьируясь от 2.75 на датасете flight до 13.75 на датасете digits (среднее значение ускорения — 6.19).

В таблице 6 представлено потребление памяти различными реализациями алгоритма FastDD на разных датасетах. Память, потребляемая алгоритмами, измерялась с помощью команды `usr/bin/time`, в качестве результата бралось максимальное потребление памяти (значение "maximum resident set size"). Все представленные показатели потребления памяти в таблице являются усреднёнными значениями по пяти

Таблица 6: Потребляемая память (Гб)

Датасет	Откр.	Перв.	Опт.
adult	4.156	1.655	**0.218**
atom	0.806	0.041	**0.041**
claim	0.539	0.080	**0.032**
digits	1.995	1.241	**0.248**
fd_reduced	1.357	0.546	**0.161**
flight	0.681	0.030	**0.030**
letter	0.933	0.726	**0.566**
neighbors	0.651	0.058	**0.058**
spstock	0.571	0.056	**0.052**
tax	0.559	0.103	**0.031**

замерам. Из таблицы можно увидеть, что оптимизированная реализация в Desbordante потребляет значительно меньше памяти, чем первоначальная и открытая реализации. Число раз, в которое уменьшается потребление памяти, варьируется от 1.65 на датасете letter до 22.70 на датасете flight (среднее значение — 13.66).

На рисунке 6 показано время работы различных стадий открытой, первоначальной и оптимизированной реализаций алгоритма. Из данного рисунка можно увидеть, что на разных датасетах большую часть времени занимают разные стадии. При этом наибольшее ускорение достигается на тех датасетах, у которых большую часть времени работы занимает стадия поиска и минимизации вершинных покрытий.

4.2. Зависимость времени работы от числа строк и атрибутов

На рисунке 7 показано время работы оптимизированной реализации алгоритма в зависимости от числа строк в датасете. Замеры производились на датасете fd_reduced с 15 атрибутами и различным числом строк. Из рисунка можно увидеть, что время работы стадии построения множества различий значительно увеличивается при увеличении числа строк, в то время как время работы стадии поиска и минимизации вершинных покрытий практически не увеличивается.

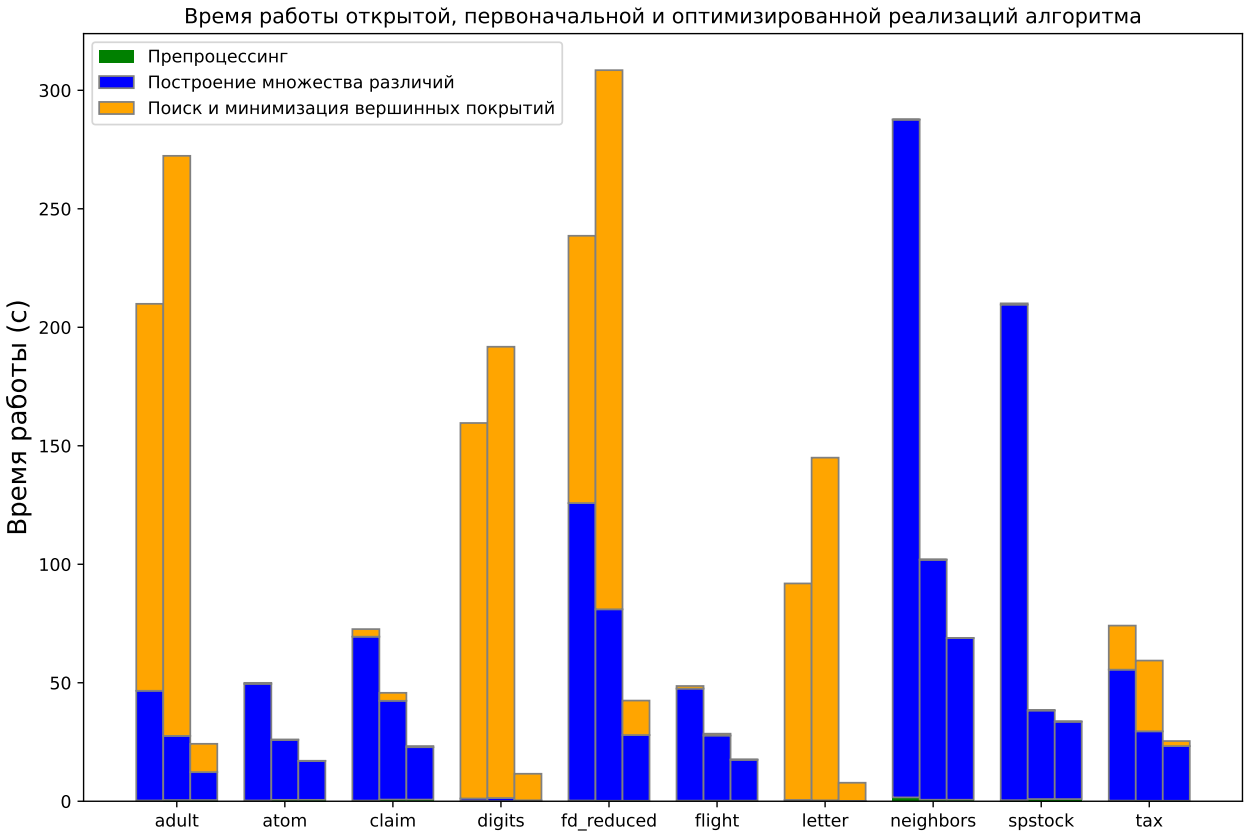


Рис. 6: Время работы различных реализаций алгоритма FastDD

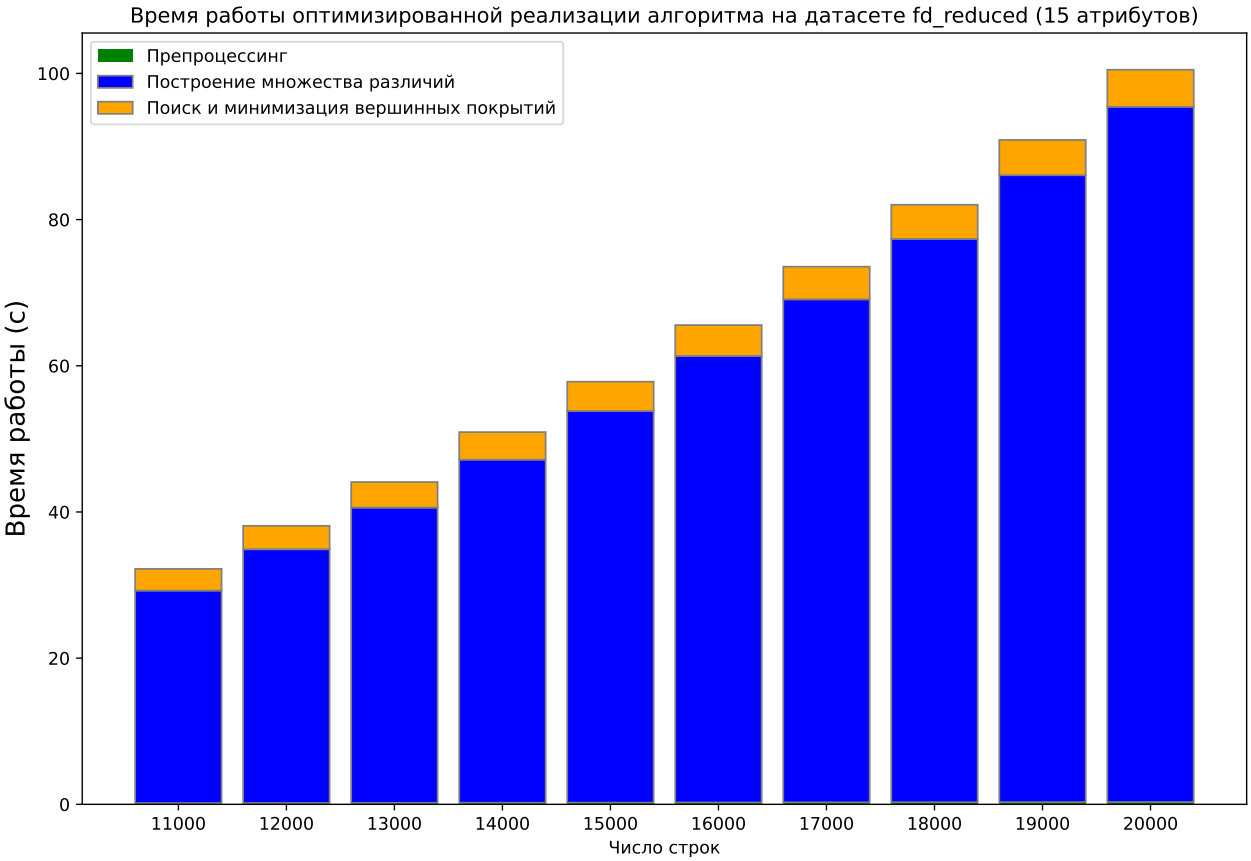


Рис. 7: Время работы FastDD в зависимости от числа строк

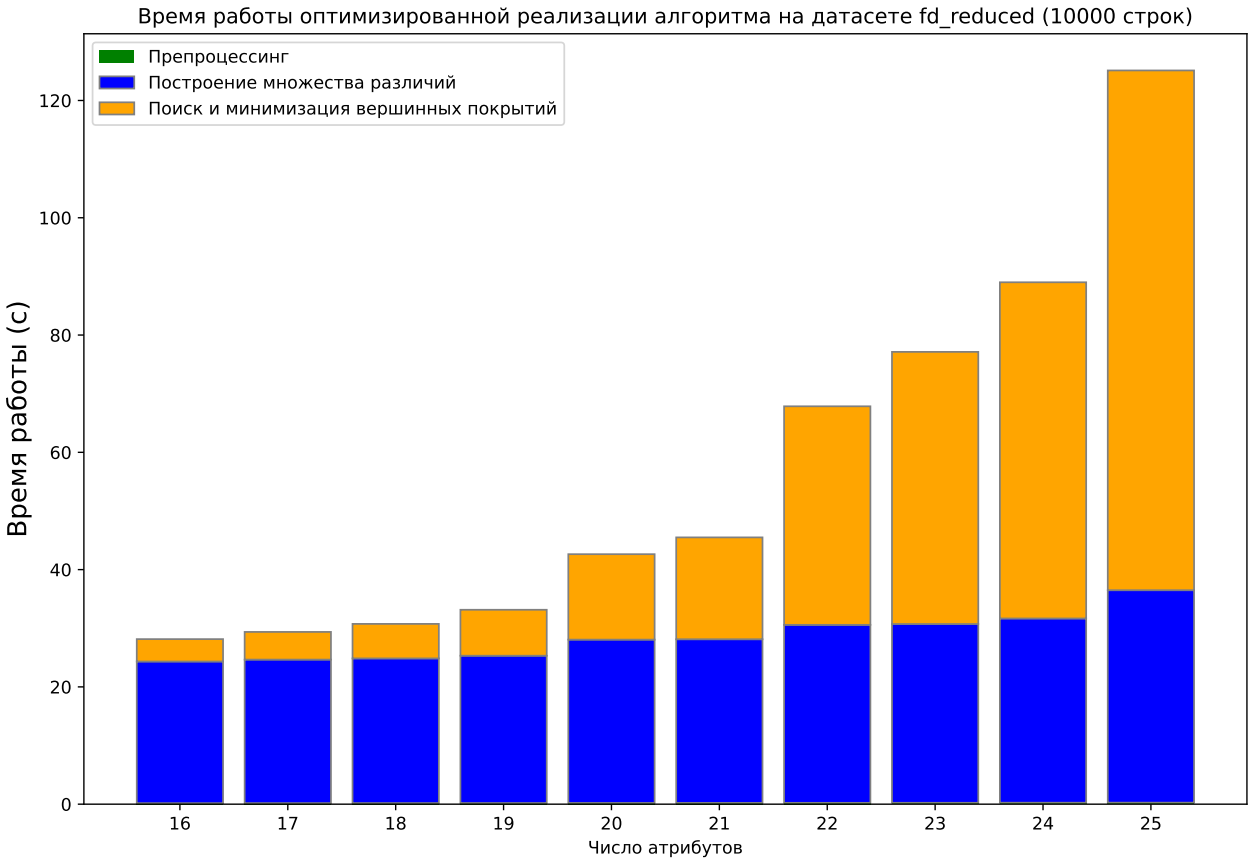


Рис. 8: Время работы FastDD в зависимости от числа атрибутов

Таблица 7: Время работы стадии поиска и минимизации вершинных покрытий (с)

Датасет	MMCS	Inv.	Inv./MMCS
adult	**11.964**	37.462	3.13×
digits	**11.193**	54.846	4.90×
fd_reduced	**14.560**	43.361	2.98×
letter	**7.665**	41.510	5.42×
tax	**2.145**	4.109	1.92×

На рисунке 8 показано время работы оптимизированной реализации алгоритма в зависимости от числа атрибутов в датасете. Замеры производились на датасете fd_reduced с 10000 строками и различным числом атрибутов. Из рисунка видно, что время работы стадии построения множества различий почти не увеличивается при увеличении числа атрибутов, в то время как время работы стадии поиска и минимизации вершинных покрытий увеличивается значительно.

Таблица 8: Время работы стадии поиска и минимизации вершинных покрытий (с)

Датасет	Фикс.	Дин.	Дин./Фикс.
adult	**11.964**	30.423	2.54×
digits	**11.193**	48.943	4.37×
fd_reduced	**14.560**	31.482	2.16×
letter	**7.665**	20.274	2.65×
tax	**2.145**	4.248	1.98×

4.3. Вклад оптимизаций алгоритма

В таблице 7 показано время работы стадии поиска и минимизации вершинных покрытий для двух стратегий: оригинальной стратегии (колонка 'Inv.') и стратегии, основанной на алгоритме MMCS (колонка 'MMCS'). Из таблицы видно, что стратегия, основанная на алгоритме MMCS, является значительно более производительной. Ускорение варьируется от 1.92 на датасете tax до 5.42 на датасете letter (среднее ускорение — 3.67).

В таблице 8 показано время работы стадии поиска и минимизации вершинных покрытий с использованием битсетов с фиксированным размером (колонка 'Фикс.') и с динамическим размером (колонка 'Дин.'). Из таблицы можно увидеть, что использование битсетов с фиксированным размером позволяет значительно ускорить работу алгоритма. Ускорение варьируется от 1.98 на датасете tax до 4.37 на датасете digits (среднее ускорение — 2.74).

На рисунке 9 показана зависимость времени работы оптимизированной реализации алгоритма FastDD от размера блока. Из графика видно, что 800 строк является оптимальным размером блока для большинства представленных датасетов.

4.4. Выводы

Экспериментами было показано, что оптимизированная реализация алгоритма FastDD в Desbordante является значительно более производительной, чем открытая реализация, причём как стадия построения

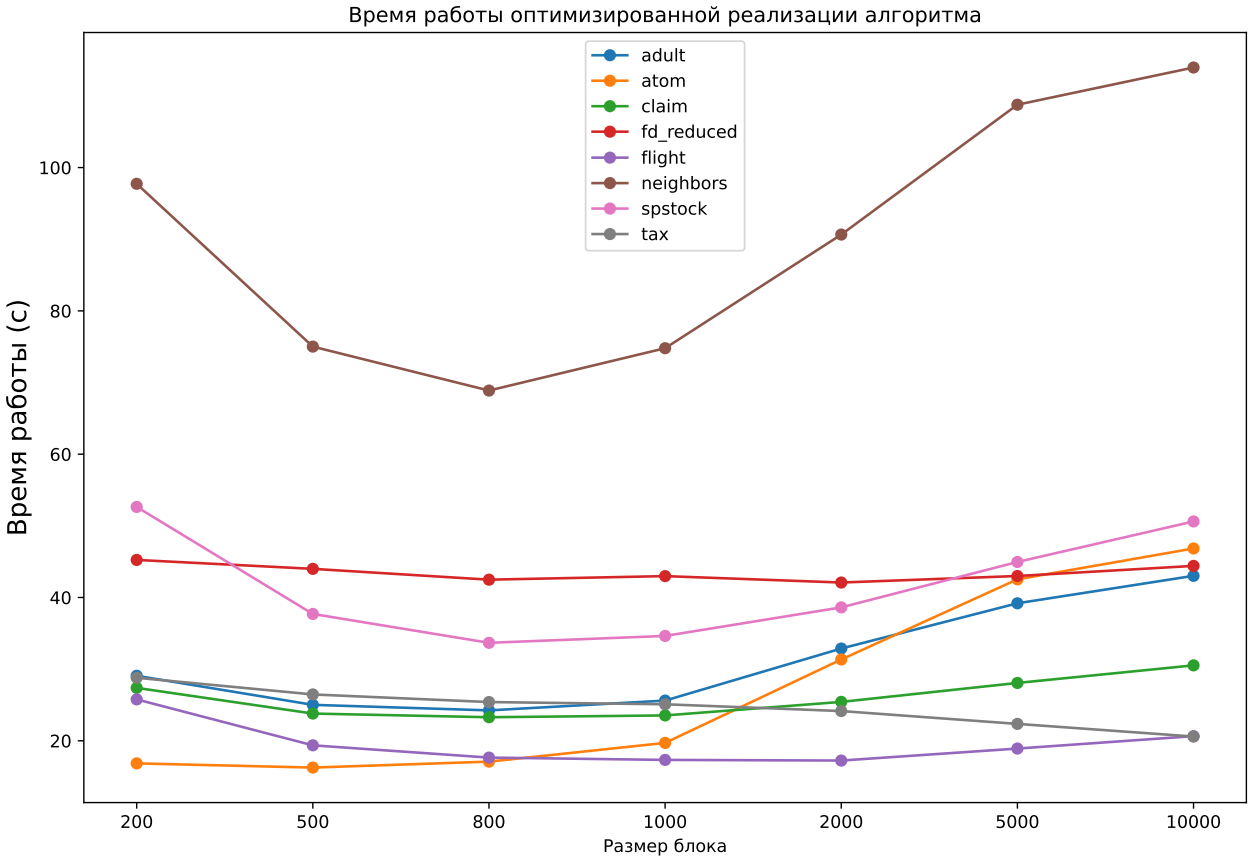


Рис. 9: Время работы FastDD в зависимости от размера блока

множества различий, так и стадия поиска и минимизации вершинных покрытий были существенно ускорены. Ускорение варьируется от 2.75 до 13.75 (среднее значение — 6.19). Кроме того, оптимизированная реализация алгоритма потребляет гораздо меньше памяти, чем открытая реализация. Число раз, в которое снижается потребление памяти, варьируется от 1.65 до 22.70 (среднее значение — 13.66).

Также было показано, что время работы стадии построения множества различий зависит в большей степени от числа строк в таблице, в то время как время работы стадии поиска и минимизации вершинных покрытий зависит больше от числа атрибутов. Помимо этого, была продемонстрирована обоснованность предложенных оптимизаций алгоритма: использования алгоритма MMCS, использования битсетов с фиксированным размером, а также подбора оптимального размера блока.

Заключение

В ходе работы были выполнены следующие задачи:

- сделан обзор алгоритма FastDD и его открытой реализации;
- алгоритм FastDD был реализован в рамках Desbordante;
- были реализованы улучшения и оптимизации алгоритма;
- было реализовано расширение алгоритма для поддержки поиска DD согласно более общему определению;
- произведено сравнение производительности открытой реализации алгоритма FastDD и его реализации в Desbordante.

Исходный код первоначальной и оптимизированной реализаций алгоритма в Desbordante доступен на GitHub[10,11].

[10] https://github.com/MichaelS239/Desbordante/tree/fastdd-baseline
[11] https://github.com/Desbordante/desbordante-core/pull/767

Список литературы

[1] Abedjan Ziawasch, Golab Lukasz, Naumann Felix. Profiling relational data: a survey // The VLDB Journal. — 2015. — aug. — Vol. 24, no. 4. — P. 557–581. — URL: https://doi.org/10.1007/s00778-015-0389-y.

[2] Desbordante: a Framework for Exploring Limits of Dependency Discovery Algorithms / Maxim Strutovskiy, Nikita Bobrov, Kirill Smirnov, George Chernishev // 2021 29th Conference of Open Innovations Association (FRUCT). — 2021. — P. 344–354.

[3] Desbordante: from benchmarking suite to high-performance science-intensive data profiler / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: https://doi.org/10.1145/3703323.3703725.

[4] Efficient Differential Dependency Discovery / Shulei Kuang, Honghui Yang, Zijing Tan, Shuai Ma // Proc. VLDB Endow. — 2024. — may. — Vol. 17, no. 7. — P. 1552–1564. — URL: https://doi.org/10.14778/3654621.3654624.

[5] Efficient Discovery of Differential Dependencies Through Association Rules Mining / Selasi Kwashie, Jixue Liu, Jiuyong Li, Feiyue Ye // Databases Theory and Applications / Ed. by Mohamed A. Sharaf, Muhammad Aamir Cheema, Jianzhong Qi. — Cham : Springer International Publishing, 2015. — P. 3–15.

[6] Functional dependency discovery: an experimental evaluation of seven algorithms / Thorsten Papenbrock, Jens Ehrlich, Jannik Marten et al. // Proc. VLDB Endow. — 2015. — jun. — Vol. 8, no. 10. — P. 1082–1093. — URL: https://doi.org/10.14778/2794367.2794377.

[7] Hitting set enumeration with partial information for unique column combination discovery / Johann Birnick, Thomas Bläsius, Tobias Friedrich et al. // Proc. VLDB Endow. — 2020. — jul. — Vol. 13, no. 12. — P. 2270–2283. — URL: https://doi.org/10.14778/3407790.3407824.

[8] Kruse Sebastian, Naumann Felix. Efficient discovery of approximate dependencies // Proc. VLDB Endow. — 2018. — mar. — Vol. 11, no. 7. — P. 759–772. — URL: https://doi.org/10.14778/3192965.3192968.

[9] Levenshtein V. I. Binary codes capable of correcting deletions, insertions, and reversals // Soviet physics doklady. — Vol. 10. — 1966. — P. 707–710.

[10] Murakami Keisuke, Uno Takeaki. Efficient algorithms for dualizing large-scale hypergraphs // Discrete Appl. Math. — 2014. — jun. — Vol. 170. — P. 83–94. — URL: https://doi.org/10.1016/j.dam.2014.01.012.

[11] Song Shaoxu, Chen Lei. Differential Dependencies: Reasoning and Discovery // ACM Trans. Database Syst. — 2011. — aug. — Vol. 36, no. 3. — 41 p. — URL: https://doi.org/10.1145/2000824.2000826.

[12] Синельников Михаил. Интеграция алгоритма поиска UCC в рамках платформы Desbordante. — 2024. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/HPIValidintegration-MichaelSinelnikov-2024spring.pdf.

[13] Синельников Михаил. Реализация алгоритма поиска дифференциальных зависимостей в рамках платформы Desbordante. — 2024. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/SPLIT-MichaelSinelnikov-2023autumn.pdf.

[14] Синельников Михаил. Обзор алгоритма FastDD поиска дифференциальных зависимостей. — 2025. — URL: https:

//github.com/Desbordante/desbordante-core/blob/main/docs/papers/FastDDreview-MichaelSinelnikov-2025spring.pdf.

[15] Синельников Михаил. Оптимизация алгоритма SPLIT поиска дифференциальных зависимостей в рамках платформы Desbordante. — 2025. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/SPLIToptimization-MichaelSinelnikov-2024autumn.pdf.